Let's create a basic action that creates an SMS message. (In the next chapter, we'll connect our app to a mock SMS server that sends and receives Twilio-compatible SMS messages.)

Add the following functions to the MessageController:

phone_app/lib/phone_app_web/controllers/message_controller.ex

```
def new(conn, params) do
  render(conn, "new.html", changeset: changeset(params))
end
```

phone_app/lib/phone_app_web/controllers/message_controller.ex

```
def create(conn, params) do
  create_changeset = changeset(params)

  case Ecto.Changeset.apply_action(create_changeset, :insert) do
    {:ok, message_params} ->
      case PhoneApp.Conversations.send_sms_message(message_params) do
        {:error, err} when is_bitstring(err) ->
          conn
          |> put_flash(:error, err)
          |> new(params)

        {:ok, _result} ->
          redirect(conn, to: ~p(/messages))
      end

    {:error, changeset} ->
      render(conn, "new.html", changeset: changeset)
  end
end
```

This function first checks if the NewMessage changeset is valid or not. This checks that the phone number is correct and that all fields are included correctly. If the parameters are valid, then the send_sms_message/1 context function is called.

The controller will either render the new.html template with errors, render new.html with a flash message, or redirect to the contact conversation.

Flash is the only concept we haven't talked about already. Flash messages[17] are session-persisted messages that display on the next page load. In our app, they are rendered in the layouts/app.html.heex file.

Our app doesn't have a send_sms_message/1 function. Let's add a mock version that persists to the database but doesn't actually send an SMS message.

17. https://hexdocs.pm/phoenix/controllers.html#flash-messages